CHƯƠNG 7: TRIỂN KHAI, TÍCH HỢP VÀ MINH CHỨNG KẾT QUẢ

Phạm vi đối chiếu: mã nguồn cục bộ ngày 06/11/2026 trên nền commit 1f2bedb và các thay đổi chưa commit. Kết quả kiểm tra được dẫn từ documentation/evidence/verification.json cùng các hồ sơ thành phần; hash của các tệp được chọn đã được đối chiếu. Đây không phải một Git tag phát hành đã niêm phong. HTTP demo 5/5, HTTP bảo mật 8/8 và ảnh runtime được ghi trước khi đồng bộ nhánh, trên nền 809f9bc; chưa chạy lại sau đồng bộ. Hồ sơ kiểm tra trước đồng bộ được giữ tại verification-before-sync.json. Số liệu 86/86 của baseline v12 ngày 28/10/2026 chỉ được giữ làm lịch sử, không thay cho kết quả hiện tại.

7.1 Tích hợp hệ thống

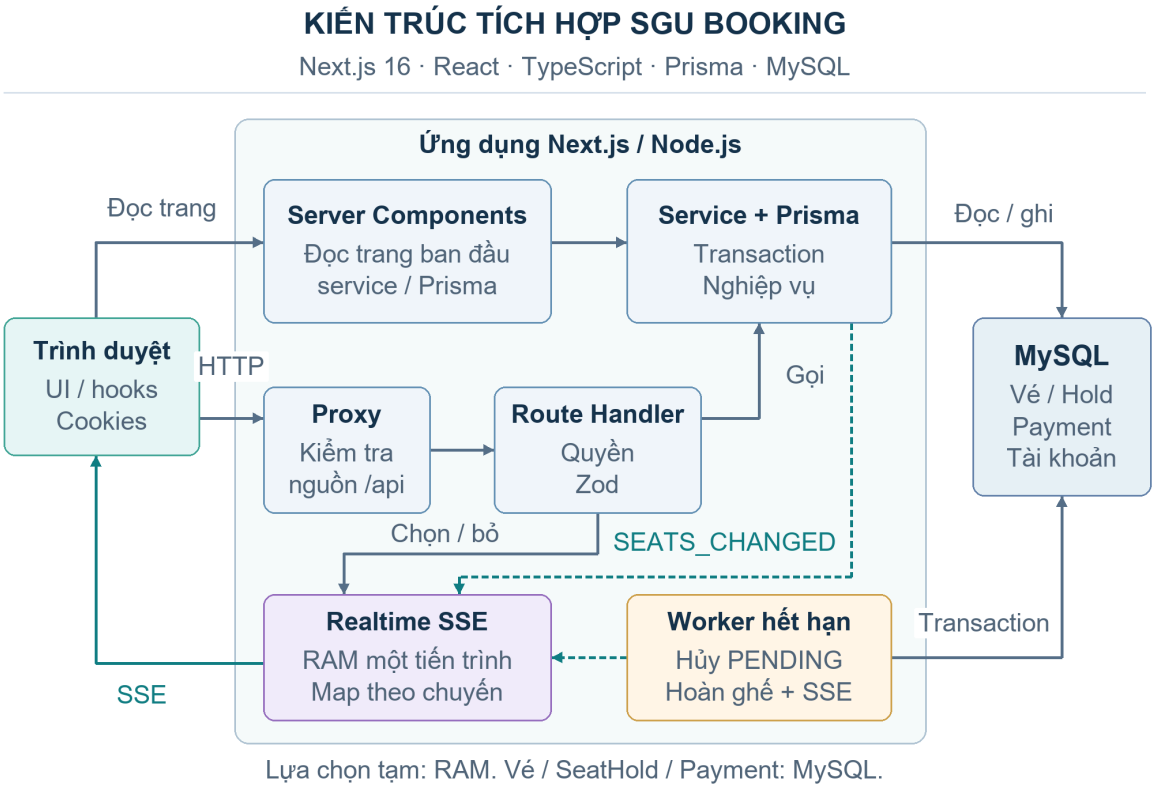
7.1.1. Kiến trúc sản phẩm và ranh giới xử lý

Hệ thống được tổ chức dưới dạng ứng dụng web full stack dùng Next.js App Router, React và TypeScript. Prisma thực hiện truy cập cơ sở dữ liệu MySQL. Trang đọc dữ liệu ban đầu ở máy chủ; các phần cần trạng thái và tương tác, như bộ lọc, biểu mẫu, sơ đồ ghế và bảng quản trị, được đặt trong Client Component. Phân chia này thể hiện trực tiếp ở trang tìm chuyến: page gọi service lấy dữ liệu, sau đó truyền initialTrips cho TripFilter xử lý tương tác tại trình duyệt.

Luồng đọc trang và luồng tương tác có các điểm vào khác nhau. Server Component có thể gọi trực tiếp service hoặc Prisma để kết xuất dữ liệu phía máy chủ. Yêu cầu HTTP từ trình duyệt đến app/api đi qua proxy.ts để kiểm tra nguồn đối với mutation và preflight, sau đó Route Handler kiểm tra schema, phiên và quyền trước khi gọi service. Proxy không thay thế guard phân quyền trong layout, page hoặc API; dữ liệu đọc ở server cũng không phải đi vòng qua HTTP nội bộ.

Thành phần	Vai trò trong sản phẩm	Tệp đối chiếu tiêu biểu
Trang phía máy chủ	Lấy dữ liệu chuyến/giá/ghế và kiểm tra phiên khi đọc trang riêng tư	app/(client)/page.tsx; app/(client)/trips/[id]/page.tsx; app/(client)/bookings/page.tsx
Giao diện và hooks	Bộ lọc URL, form, chọn ghế, tải lại và thông báo lỗi	components/trips/SeatSelector.tsx; hooks/useSeatRealtime.ts; hooks/useRemoteData.ts
Proxy và API HTTP	Kiểm tra origin trước mutation; handler kiểm tra schema, phiên và quyền	proxy.ts; app/api/bookings/route.ts; app/api/admin/orders/[id]/route.ts
Service nghiệp vụ	Đặt/hủy/xác nhận vé, tổng tiền, transaction và chuẩn hóa dữ liệu	services/bookingService.ts; services/orderService.ts; services/paymentService.ts; lib/transaction.ts
Realtime và worker	Đồng bộ ghế/người xem; hủy PENDING quá hạn và trả ghế	lib/sse.ts; hooks/useSeatRealtime.ts; lib/booking-expiry-worker.ts; services/bookingExpiryService.ts
Dữ liệu và ràng buộc	Prisma/MySQL lưu vé, chuyến, tài khoản và thanh toán; unique/FK/CHECK	prisma/schema.prisma; prisma/admin-migrations/; scripts/demo.cjs
Tái lập và đặc tả	Dựng demo riêng; sinh/kiểm tra OpenAPI và trang tài liệu	scripts/demo.cjs; scripts/generate-api-docs.cjs; documentation/api/

Bảng 7.1. Các lớp tích hợp và vị trí mã nguồn



Hình 7.1. Kiến trúc tích hợp theo mã nguồn hiện tại: đọc trang phía server, API có kiểm tra nguồn/quyền, SSE trong một tiến trình và worker xử lý vé quá hạn. Đây là sơ đồ kỹ thuật, không phải ảnh runtime.

7.1.2. Quan hệ kiến thức → quyết định → sản phẩm

Việc tích hợp được đánh giá bằng nơi kỹ thuật xuất hiện trong sản phẩm và tác dụng của nó đối với luồng nghiệp vụ. Các nhận định về tái sử dụng và tổ chức lại dưới đây căn cứ vào cấu trúc đang tồn tại trong repository.

Kiến thức/cơ chế	Quyết định hiện thực	Kết quả trong sản phẩm
Server/Client Component	Đọc ban đầu ở server; state và tương tác ở client	Trang chuyến/lịch sử gọi service; TripFilter và SeatSelector tương tác tại trình duyệt
Phân lớp nghiệp vụ	Online và tại quầy cùng gọi createBooking	Quy tắc ghế, chuyến, giá, giảm chỗ và PNR được dùng chung
Schema validation	Zod ở API; form áp dụng schema/React Hook Form khi phù hợp	Chặn input sai; server quyết định giá và trạng thái vé online
Transaction và ràng buộc	Serializable, retry giới hạn, unique activeSeat và FK/ CHECK	Tạo nhiều vé cùng PNR và giảm ghế trong một transaction
Realtime	HTTP POST/PUT gửi thao tác; SSE và polling tải lại trạng thái	Đồng bộ lựa chọn, bookedSeats và viewer trong phạm vi một tiến trình
Vòng đời vé	Lưu expiresAt; worker cập nhật có điều kiện	Hủy vé online quá hạn, trả kho ghế đúng một lần, giữ lịch sử
Xác thực và bảo mật	Scrypt, phiên ký số, guard ADMIN, origin và output escaping	Bảo vệ trang/API; kiểm tra XSS/CORS và ghi rõ phạm vi OWASP

Kiến thức/cơ chế	Quyết định hiện thực	Kết quả trong sản phẩm
Logic giao diện dùng chung	Client service, hooks, hủy request, debounce 350 ms	Các danh sách quản trị dùng chung tải dữ liệu, hành động và xử lý lỗi
Khả năng tái lập	Database demo có marker sở hữu; không reset dữ liệu có sẵn	Một lệnh tạo seed và chạy; có ADMIN/ CUSTOMER để thử chức năng
Hợp đồng API	Sinh từ AST, Zod và metadata; check trước build	OpenAPI 3.1.1, trang /api-docs và endpoint /api/openapi

Bảng 7.2. Từ cơ chế đã học đến quyết định và chức năng sản phẩm

7.1.3. Tích hợp luồng đặt vé và bảo toàn dữ liệu

Luồng đặt vé trực tuyến bắt đầu từ tìm chuyến, mở chi tiết, chọn ghế và nhập thông tin hành khách. SeatSelector gửi POST /api/bookings; API kiểm tra onlineOrderSchema và yêu cầu createBooking tạo vé PENDING. Với phiên CUSTOMER hợp lệ, ownerId lấy từ phiên. Khách chưa đăng nhập vẫn được đặt vé; service tạo tài khoản kỹ thuật USER riêng có định danh guest ngẫu nhiên, không gán quyền sở hữu tài khoản hay lịch sử cũ chỉ dựa vào số điện thoại hành khách nhập vào.

Trong services/bookingService.ts, mã ghế được chuẩn hóa; mỗi lượt phải có 1–5 ghế khác nhau. Transaction đọc chuyến, từ chối chuyến đã xuất bến, kiểm tra ghế thuộc sơ đồ capacity, vé chưa hủy và SeatHold còn hạn. Số ghế trống chỉ giảm khi availableSeats còn ít nhất bằng số ghế cần đặt. Lựa chọn tạm trong RAM phục vụ giao diện; service vẫn kiểm tra trạng thái database khi ghi vé, không dùng quyền lựa chọn trong RAM làm bảo đảm mua ghế.

Mỗi ghế được lưu thành một bản ghi booking với giá lấy từ trip.price, thông tin hành khách riêng và activeSeat có dạng <tripId>:<seatNumber>. Các vé cùng lượt dùng chung PNR được sinh bằng dữ liệu ngẫu nhiên; các giữ ghế liên quan được xóa sau khi tạo vé. Việc giảm ghế, tạo vé và gán PNR cùng nằm trong transaction nên không được công bố thành công khi chỉ một phần thao tác ghi hoàn tất.

lib/transaction.ts dùng mức cô lập Serializable và thử lại tối đa ba lần đối với lỗi P2034; lỗi khác được trả ngay. Datamodel có unique activeSeat và khóa ngoại; SQL nghiệp vụ bổ sung CHECK sức chứa, trạng thái và tiền. Hồ sơ demo đã kiểm tra các ràng buộc trên MySQL 8.0.46 trong database riêng. Snapshot demo đổi riêng FK booking.tripId sang ON UPDATE RESTRICT để tương thích CHECK; kết quả đó không chứng minh chuỗi migration lịch sử đã chạy thành công trên mọi database.

Vé online PENDING chiếm ghế ngay khi tạo và có expiresAt là thời điểm sớm hơn giữa sau 5 phút và giờ xuất bến. instrumentation.ts khởi động worker trong Node runtime, bỏ qua giai đoạn build. Worker chạy lượt đầu sau khoảng 1 giây, chờ 15 giây sau mỗi lượt và xử lý tối đa 100 vé quá hạn theo thứ tự expiresAt/id. Transaction cập nhật có điều kiện PENDING thành CANCELLED, xóa activeSeat và trả số ghế đúng một lần, rồi phát thông báo làm mới ghế. Đây là hoàn kho ghế, không phải hoàn tiền; vé có expiresAt = null không tự hết hạn. Việc giải phóng không được cam kết đúng tại giây thứ 300 và cần server đang chạy.

7.1.4. Tích hợp realtime và ranh giới giữ ghế

Trước khi mở realtime, useSeatRealtime gọi GET /api/trips/{id}/holds?clientId=... để nhận cookie seat_session và tải capacity, bookedSeats, giữ ghế còn hạn cùng lựa chọn tạm. Hook sau đó mở GET /api/trips/{id}/stream bằng EventSource. SEAT_SNAPSHOT có selections, bookedSeats và viewerCount; các sự kiện tiếp theo gồm SEAT_SELECTED, SEAT_DESELECTED, VIEWERS_UPDATED và SEATS_CHANGED. Server gửi heartbeat mỗi 15 giây và dọn kết nối khi phát hiện stream đóng hoặc bị hủy.

POST /seat-events gửi thao tác chọn/bỏ chọn; PUT /seat-events đồng bộ danh sách sau khi mở hoặc phục hồi kết nối. Giao diện cập nhật lạc quan, hoàn tác và kết nối lại khi yêu cầu thất bại. Server kiểm tra tripId, ghế, chuyến, kết nối đang hoạt động và khóa sở hữu suy ra từ cookie HttpOnly đã ký cùng clientId. Biết clientId công khai không đủ để thao tác thay trình duyệt khác. Khi nối lại, hook tải trạng thái hiện tại, loại ghế không còn khả dụng rồi mới đồng bộ; chưa có hàng đợi thao tác offline.

Trạng thái chọn tạm nằm trong globalThis của một tiến trình Node.js, chưa chia sẻ giữa nhiều instance và mất khi tiến trình khởi động lại. Lựa chọn này không có thời hạn giữ ghế 5 phút và không tạo quyền mua được bảo đảm trong database. Số người xem được đếm theo clientId khác nhau trong các kết nối của từng chuyến, không phải số tài khoản đăng nhập.

SeatHold là cơ chế riêng: POST /api/seats/hold lưu giữ ghế có hạn 5 phút trong MySQL, gắn với khóa sở hữu của phiên. SeatSelector chưa tự gọi API này khi bấm ghế, nhưng hook đã đọc /holds để hiển thị và loại ghế đang được phiên khác giữ. Do đó, chọn ghế realtime, tạo SeatHold và vé PENDING có thời hạn thanh toán là ba trạng thái khác nhau. createBooking kiểm tra lại vé và giữ ghế trong database; clientId trong body đặt vé vẫn là trường tùy chọn.

Sau khi tạo vé, đổi/hủy trạng thái qua quản trị hoặc hủy vé quá hạn, server phát SEATS_CHANGED sau khi transaction hoàn tất. Hook nhận sự kiện sẽ tải lại bookedSeats, holds và capacity; đồng thời kiểm tra định kỳ mỗi 5 giây để bù sự kiện bị lỡ. PENDING và CONFIRMED đều chiếm ghế, nên xác nhận thanh toán không trừ ghế thêm lần nữa. SSE hỗ trợ cập nhật giao diện; transaction và ràng buộc database vẫn quyết định việc ghi nhận vé hợp lệ.

7.1.5. Tích hợp xác thực, quản trị, bảo mật và thanh toán

Đăng ký tạo dữ liệu user và customer trong transaction, chuẩn hóa thông tin và tự thiết lập phiên đăng nhập. Mật khẩu được hash bằng scrypt với salt ngẫu nhiên. Token phiên có HMAC-SHA256, thời hạn 24 giờ, nonce và thành phần gắn với mật khẩu hiện tại. Cookie được thiết lập HttpOnly, SameSite=Lax và Secure khi chạy production. Khi dùng phiên, server đọc lại trạng thái hoạt động, vai trò và phiên bản mật khẩu, nhờ đó tài khoản bị khóa hoặc đổi mật khẩu không tiếp tục sử dụng phiên cũ.

Layout và API quản trị kiểm tra ADMIN độc lập với menu. Trang /bookings chỉ lấy vé có userId thuộc phiên CUSTOMER. Tra cứu công khai GET /api/bookings yêu cầu đồng thời pnr và phone hành khách khớp chính xác; giới hạn 20 lượt/điện thoại và 60 lượt/nguồn trong 60 giây, vượt mức trả 429 kèm Retry-After. Bộ đếm nằm trong từng tiến trình; nhiều worker cần bộ đếm chung và proxy xác định nguồn tin cậy. Proxy chặn mutation/preflight khác origin; ứng dụng không mở wildcard CORS và có header bảo mật. Các biện pháp này không thay thế kiểm toán bảo mật toàn hệ thống.

Thanh toán dùng ảnh VietQR Quick Link và xác nhận giả lập. Backend cộng giá vé đã lưu theo PNR, tạo nội dung chuyển khoản VE kèm ID vé đầu tiên; trình duyệt chỉ gửi PNR khi bấm xác nhận Demo. Khi PAYMENT_DEMO_ENABLED là true, service tạo Payment SUCCESS có responseCode DEMO và mã giao dịch DEMO-, đồng thời chuyển toàn bộ vé PENDING hợp lệ sang CONFIRMED, ghi confirmedAt trong transaction. Vé hết hạn, đã xuất bến hoặc không còn PENDING bị từ chối; số ghế không giảm thêm.

Service kiểm tra Payment SUCCESS/REVIEW để xử lý xác nhận lặp; schema không đặt unique trên PNR của Payment. Chưa có webhook, kiểm chứng tiền vào ngân hàng hoặc hoàn tiền tự động. PaymentPanel cập nhật trạng thái và refresh sau khi xác nhận, chưa polling trạng thái thanh toán nền. /success và API thanh toán vẫn dùng PNR, không áp dụng cặp PNR–điện thoại như API tra cứu. Doanh thu quản trị tính từ vé CONFIRMED, chưa đồng nghĩa đã đối soát tiền thật; trình diễn dùng database demo riêng.

7.2. Kết quả đạt được — minh chứng theo track

7.2.1. Phạm vi kết quả của hai track

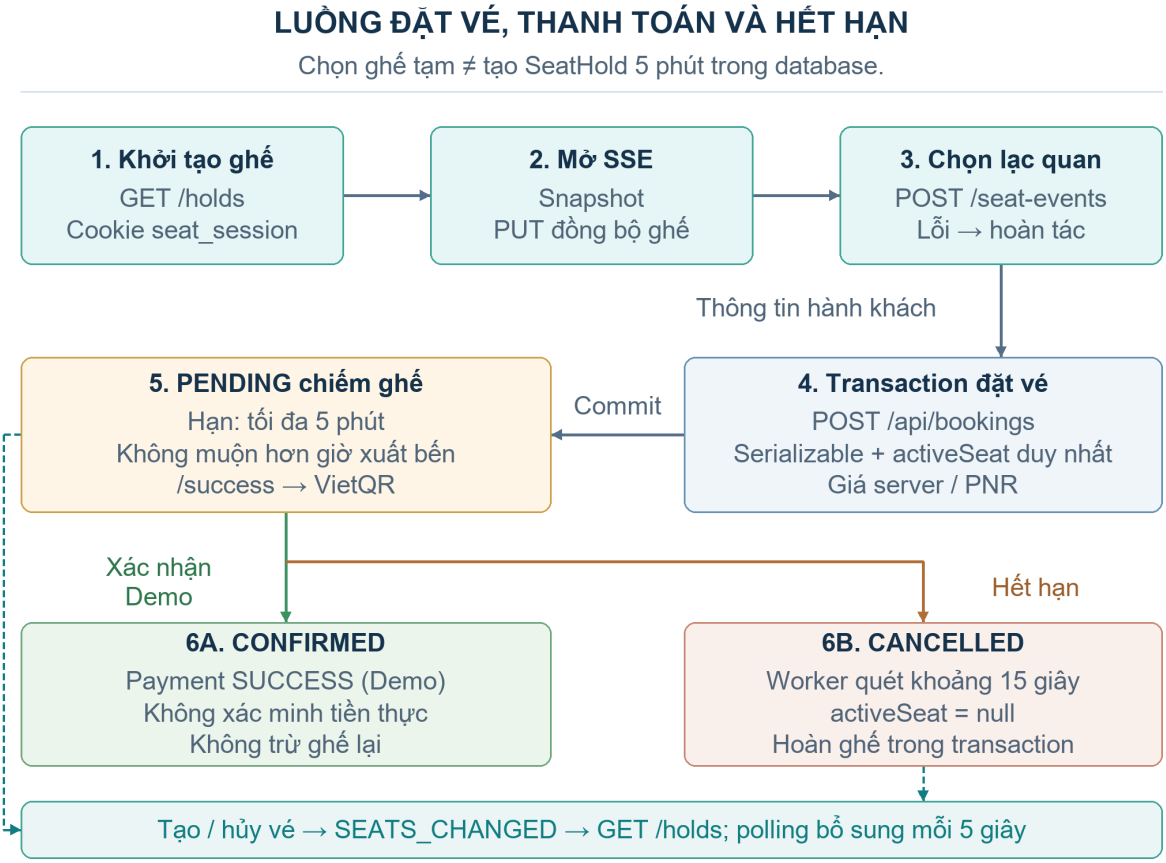
Dự án thể hiện hai track Web/Frontend và Backend/API. Chưa có module sản phẩm thuộc AI/Data hoặc nền tảng Automation/Low-code; việc dùng AI hỗ trợ viết code không được tính thành chức năng AI của hệ thống. Source repository được ghi nhận tại https://github.com/phongvuz/sgu-booking. Các kết quả dưới đây phân biệt mã nguồn đã hiện thực, kiểm thử mock, kiểm tra HTTP và database demo; địa chỉ repository không phải bằng chứng một site công khai đã triển khai.

Hạng mục	Kết quả thể hiện trong mã nguồn	Minh chứng và giới hạn
Tìm chuyến	Dữ liệu từ DB; lọc ngày/giờ UTC+7, xe theo capacity trước phân trang	Unit bộ lọc; ảnh demo trước đồng bộ (809f9bc); chưa có quan hệ xe–chuyến riêng
Chọn ghế/đặt vé	Sơ đồ capacity; cookie sở hữu, SSE, đặt 1–5 ghế chung PNR	Test logic/handler; chưa có kết quả cuộc đua đặt vé trên MySQL trong hồ sơ
Hết hạn	Online PENDING tối đa 5 phút; worker hủy/hoàn ghế và phát cập nhật	Code worker/service; chưa có phép đo SLA hoặc E2E chờ hết hạn
Tài khoản khách	Đăng ký/đăng nhập/hồ sơ; lịch sử theo phiên, tra cứu PNR + điện thoại	Unit/handler hiện tại; HTTP demo trước đồng bộ xác minh login/session/ profile và từ chối quyền admin
Quản trị đơn	Danh sách/chi tiết/tạo/đổi trạng thái/hủy đã nối API và service	Test API và nghiệp vụ với mock; DELETE hủy giữ lịch sử, mỗi ID là một vé/ghế

Hạng mục	Kết quả thể hiện trong mã nguồn	Minh chứng và giới hạn
Quản trị dữ liệu	Quản lý chuyến, xe, nhân viên, tài khoản, đơn, thanh toán và thống kê	Unit lọc/sort/guard/thống kê; chưa chạy toàn bộ browser suite
Thanh toán	QR theo dữ liệu vé; xác nhận Demo atomically và không trừ ghế lại	payment.test.cjs; không xác minh nhận tiền ngân hàng
Tái lập/dữ liệu	Database demo riêng; seed, bảo toàn khi chạy lại, CHECK/FK/unique	demo-db.cjs đạt trên MySQL 8.0.46; không chứng minh lịch sử migration
Bảo mật/API docs	OWASP nhận biết, origin/header/XSS; OpenAPI sinh lại cho 28 đường dẫn, 45 thao tác	HTTP 8/8 và HTML/JSON 200 thuộc nền 809f9bc trước đồng bộ; chưa thử đủ 45 nghiệp vụ
Chất lượng mã	162/162 unit/handler; lint, TypeScript và production build đạt	Log pre-push ngày 06/11/2026, nền 1f2bedb; không có số đo tải, độ bao phủ hoặc WCAG

Bảng 7.3. Kết quả theo chức năng và phạm vi minh chứng

7.2.2. Luồng chức năng chính và cách đối chiếu



Hình 7.2. Luồng đặt vé hiện tại và hai nhánh xử lý PENDING: xác nhận Demo hoặc worker hủy quá hạn. SSE đồng bộ lựa chọn và báo thay đổi ghế; transaction là điểm ghi nhận vé.

Luồng khách đặt vé được đối chiếu theo sáu bước sau:

1. Người dùng chọn điểm đi, điểm đến và ngày trên trang chủ. SearchWidget gửi GET /trips; server chuẩn hóa bộ lọc, truy vấn chuyến chưa xuất bến và còn ghế rồi truyền dữ liệu ban đầu cho TripFilter. Bộ lọc giờ/ngày áp dụng theo UTC+7 trước phân trang; bộ lọc loại xe hiện quy ước từ capacity, không suy đoán bằng số ghế trống.

2. Khi mở /trips/{id}, server kiểm tra ID nguyên dương và đọc chuyến, sức chứa cùng ghế đã có vé. Hook tải /holds, thiết lập cookie phiên chọn ghế, mở SSE, nhận snapshot và đồng bộ lại lựa chọn trước khi cho thao tác. Những trạng thái này được kiểm tra lại khi có sự kiện, theo chu kỳ và khi kết nối phục hồi.

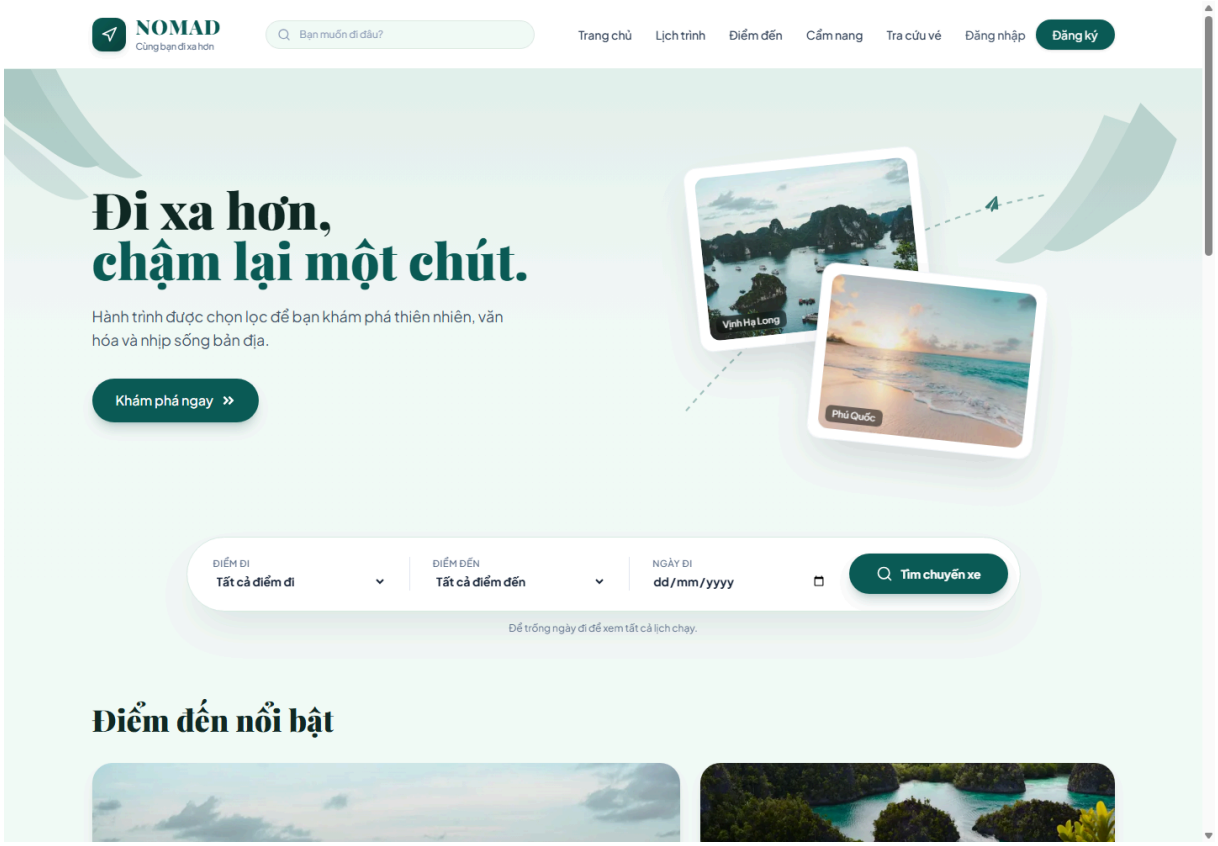
3. Người dùng chọn tối đa 5 ghế và nhập thông tin hành khách. Thao tác chọn/bỏ chọn được gửi qua HTTP và phát lại cho các kết nối cùng chuyến. Lựa chọn của người khác được hiển thị riêng trên sơ đồ.

4. POST /api/bookings được kiểm tra lại tại server. Khi transaction thành công, API trả HTTP 201 cùng pnr, tripId, passenger, seats, totalPrice và bookingIds. Nếu ghế đã có vé hoặc không đủ chỗ, API trả lỗi thay vì ghi nhận một phần lượt đặt.

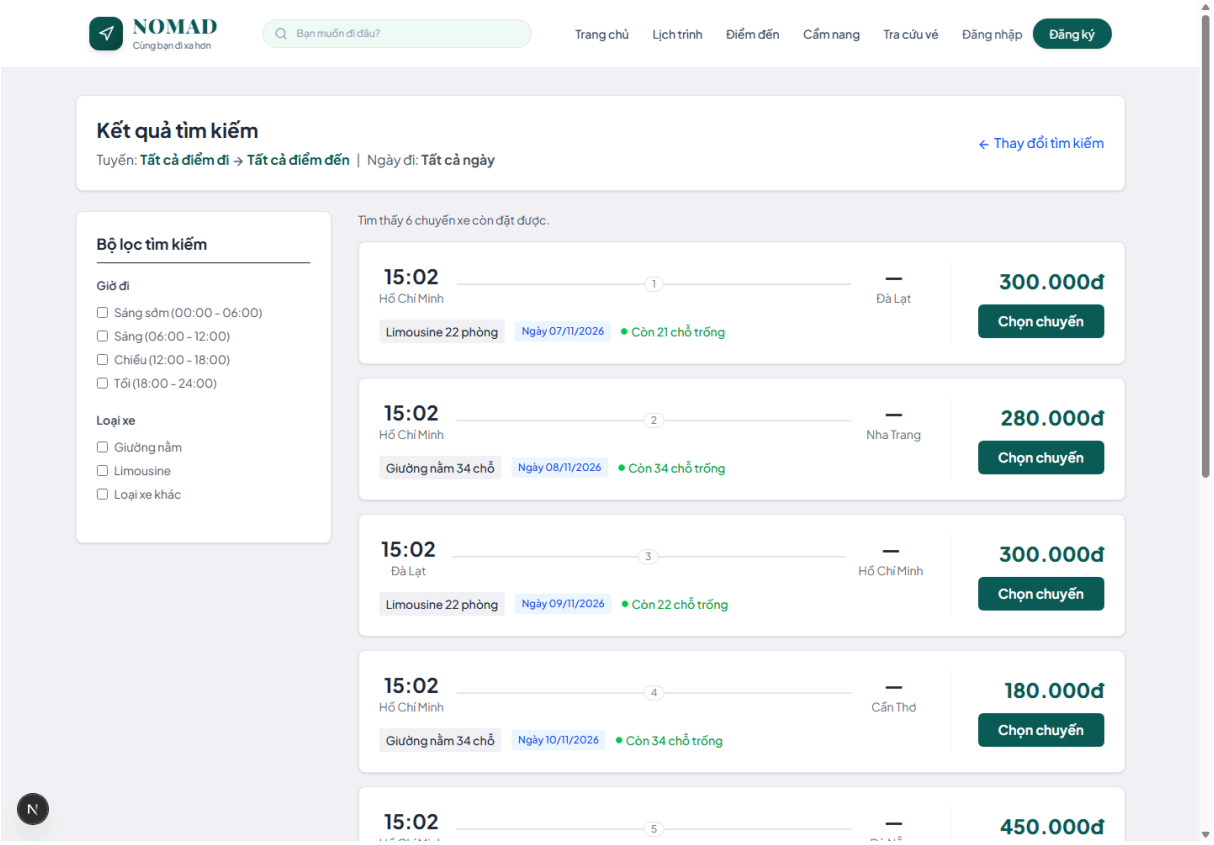
5. Trình duyệt chuyển tới /success?pnr=...; trang đọc bản ghi theo PNR để hiển thị vé, tổng tiền và PaymentPanel. Đây là kết quả lưu lượt đặt, chưa đồng nghĩa đã thu tiền. Xác nhận Demo tính tiền từ database; nếu đã hết hạn thì bị từ chối. Worker xử lý vé quá hạn độc lập với việc trình duyệt còn mở trang thanh toán.

6. Khách đăng nhập xem lịch sử tại /bookings; ADMIN xem đơn và giao dịch tại /admin/orders, /admin/payments. Khách vãng lai tra cứu bằng cả PNR lẫn số điện thoại hành khách; nhập số điện thoại không tự tạo phiên CUSTOMER hoặc chuyển lịch sử của vé vãng lai sang tài khoản vừa đăng ký.

Các endpoint đơn hàng quản trị đã nối service: GET danh sách/chi tiết, POST tạo vé tại quầy, PATCH đổi trạng thái và DELETE hủy có giữ lịch sử. Mỗi ID là một bản ghi booking tương ứng một ghế; nhiều ghế có thể chung PNR. Hủy lặp không trả ghế thêm; khôi phục kiểm tra giờ đi, thời hạn vé, sơ đồ, ghế đã có vé/đang giữ và chỗ trống. CONFIRMED không chuyển ngược sang PENDING. Modal có thể dùng object từ danh sách để hiển thị, nhưng API chi tiết vẫn đã được triển khai.



Hình 7.3. Trang chủ NOMAD chụp từ bản demo cục bộ ngày 06/11/2026, trên nền 809f9bc trước khi đồng bộ nhánh. Ảnh minh chứng kết xuất giao diện trên dữ liệu demo; không phải ảnh của một triển khai cloud.



Hình 7.4. Danh sách /trips trên demo riêng ngày 06/11/2026, nền 809f9bc trước đồng bộ, với chuyến khởi hành trong tương lai. Ảnh chỉ minh chứng luồng đọc và hiển thị, không chứng minh đã đặt hoặc thanh toán vé.

7.2.3. Đặc tả API tự sinh và cách đối chiếu

Repository hiện có OpenAPI 3.1.1 tại documentation/api/openapi.json, cung cấp qua GET /api/openapi và giao diện /api-docs. Bản được kiểm tra có 28 đường dẫn, 45 thao tác HTTP. Generator phát hiện handler bằng TypeScript AST, chuyển schema đầu vào Zod và kết hợp metadata nghiệp vụ, quyền, phản hồi. npm run api:docs sinh tài liệu; api:docs:check phát hiện bản thiếu/cũ và được chạy trước build. Refinement nhiều trường và kiểm tra database vẫn thực thi ở server; response service chưa có schema được để mở. Bảng dưới đây là trích yếu đặc tả, không tự chứng minh toàn bộ request đã được thực thi thành công.

Phương thức và đường dẫn	Đầu vào / quyền	Kết quả chính	Lỗi đáng chú ý
GET /api/trips	Công khai: from, to, date, times, vehicles, page, limit. admin=true cần ADMIN	200: chuyến còn nhận khách, phân trang; chế độ admin có stats	400 bộ lọc; 401/403 khi gọi admin=true thiếu quyền
POST /api/bookings	tripId số; seats 1–5; fullName, phone; clientId tùy chọn	201: PNR, ghế, tiền và ID vé; online PENDING do server quyết định	400 input; 404 chuyến; 409 ghế/chuyến/chỗ trống
GET /api/bookings	Bắt buộc pnr và phone khớp chính xác; công khai	200: success, count, data; không khớp trả danh sách rỗng; no-store	400 thiếu/sai input; 429 vượt giới hạn, có Retry-After

Phương thức và đường dẫn	Đầu vào / quyền	Kết quả chính	Lỗi đáng chú ý
GET /api/trips/{id}/holds	ID số; clientId để khởi tạo/đối chiếu phiên chọn ghế	200: capacity, bookedSeats, holds, selections; có thể cấp cookie	400 ID/clientId; 404 chuyến
GET /api/trips/{id}/stream	ID số; clientId hợp lệ; cookie phiên khi gắn quyền thao tác	200 text/event-stream: snapshot, viewer, sự kiện và heartbeat	400 ID/clientId; 404 chuyến
POST /api/trips/{id}/seat-events	tripId, clientId, seatNumber, type; khóa sở hữu từ cookie	200: chọn/bỏ chọn và broadcast theo chuyến	400 schema/ID; 403 nguồn; 404 chuyến; 409 xung đột/phiên
PUT /api/trips/{id}/seat-events	tripId, clientId, selectedSeats tối đa 5; cookie sở hữu	200: đồng bộ lại danh sách lựa chọn	400 dữ liệu; 403 nguồn; 404 chuyến; 409 không khả dụng
POST /api/auth/login	identifier, password, audience customer/admin	200: success, redirectTo và cookie phiên	400 input; 401 thông tin/role/trạng thái; 403 nguồn
POST /api/auth/register	fullName, email, phone, password, confirmPassword, agreeTerms; address tùy chọn	201 và cookie; user/customer cùng transaction	400 input; 403 nguồn; 409 trùng dữ liệu
POST /api/auth/logout; GET /api/auth/session	Cookie phiên; logout kiểm tra nguồn	200: xóa cookie hoặc trả authenticated/user công khai	403 logout khác nguồn; lỗi nội bộ được trả thông báo chung
GET /api/payments/status	pnr 1–191 ký tự; công khai	200: status và bookingStatus; no-store	400 PNR; 404 lượt đặt
POST /api/payments/demo/confirm	Body strict chỉ pnr; demo phải bật	200: SUCCESS/CONFIRMED; lặp hợp lệ không tạo lại	400 body; 403 nguồn/demo tắt; 404; 409; 503 QR
GET/POST /api/admin/orders	ADMIN; GET filter/page/sort; POST tripId, seats, fullName, phone, status	200 danh sách; 201 tạo vé tại quầy PENDING/CONFIRMED	400 input; 401/403 quyền; 409 nghiệp vụ
GET/PATCH/DELETE /api/admin/orders/{id}	ADMIN; id một booking; PATCH nhận trạng thái hợp lệ	200: chi tiết/cập nhật/hủy có giữ lịch sử	400 ID/input; 401/403; 404; 409 xung đột
GET /api/openapi	Công khai; không cần database	200: đặc tả OpenAPI 3.1.1; UI tương ứng /api-docs	Nguồn/quyền các API nghiệp vụ vẫn áp dụng riêng

Bảng 7.4. Trích yếu API hiện tại; đặc tả đầy đủ tại /api-docs và /api/openapi

GET/PATCH/DELETE /api/admin/orders/{id} thực hiện xem chi tiết, đổi trạng thái và hủy giữ lịch sử. GET /api/trips/{id}/holds đã được tích hợp vào hook để khởi tạo phiên, tải trạng thái và khôi phục kết nối. Riêng POST /api/seats/hold vẫn là API giữ ghế độc lập, chưa được SeatSelector tự gọi khi người dùng chọn ghế.

Ví dụ body đặt vé để tái lập sau khi thay tripId bằng chuyến đang còn nhận khách và A01 bằng ghế trống thực tế:

```
{
  "tripId": 123,
  "seats": ["A01"],
  "fullName": "Khach kiem thu",
  "phone": "0901234567"
}
```

Giá và trạng thái không nằm trong body online. Thành công trả data.totalPrice do server tính, cùng data.pnr và bookingIds. Tra cứu dùng GET /api/bookings?pnr=<PNR>&phone=<SDT_HANH_KHACH>; trạng thái thanh toán dùng GET /api/payments/status?pnr=<PNR>; xác nhận Demo nhận body chỉ có pnr. Các ví dụ là cấu trúc hướng dẫn, cần thay ID/ghế bằng dữ liệu thật của database demo; hồ sơ HTTP hiện có chưa ghi nhận một lượt đặt–thanh toán end-to-end hoàn chỉnh.

API quản trị yêu cầu đăng nhập ADMIN và cookie do server cấp; API khách hàng yêu cầu CUSTOMER khi nghiệp vụ quy định. Có thể dùng công cụ HTTP có cookie jar để lưu response và dữ liệu trước–sau, nhưng chưa có kết quả Postman Collection Runner trong hồ sơ. Tài liệu không chứa token thật, SESSION_SECRET hoặc thông tin ngân hàng sử dụng thực tế.

7.2.4. Kết quả kiểm tra đã thực hiện

Các kết quả ngày 06/11/2026 được ghi trên Windows, Node.js v24.22.1 và npm 11.19.1. Bộ unit/handler và kiểm tra chất lượng trước khi push áp dụng cho nền 1f2bedb cùng các thay đổi cục bộ; mã nguồn tương ứng được đối chiếu bằng hash trong hồ sơ. Unit/handler test dùng hàm thuần, kết xuất HTML và mock phụ thuộc; bộ đầy đủ chạy cách ly từng tệp. Hồ sơ trước đồng bộ còn ghi việc tạo dữ liệu và kiểm tra ràng buộc trên demo MySQL 8.0.46, cùng các phép HTTP riêng trên nền 809f9bc. Những phép HTTP và ảnh trang chưa được chạy/chụp lại sau đồng bộ. Các phạm vi này được tách rõ trong Bảng 7.5, không dùng một loại bằng chứng thay cho toàn bộ E2E.

Mã	Lệnh / bằng chứng	Kết quả quan sát	Phạm vi kết luận
MC-01	npm test; documentation/evidence/pre-push-unit.log	162/162 đạt; 0 fail, 0 skipped; khoảng 6,30 giây	Nền 1f2bedb và thay đổi cục bộ; unit/render/handler cách ly từng tệp
MC-02	npm run typecheck; documentation/evidence/pre-push-typecheck.log	Exit code 0	TypeScript đạt theo cấu hình trước khi push
MC-03	npm run lint; documentation/evidence/pre-push-lint.log	Exit code 0	ESLint đạt theo source/cấu hình trước khi push
MC-04	npm run build với SGU_DEMO=true; documentation/evidence/pre-push-build.log; pre-push-build-result.json	Exit 0; biên dịch và tạo bản production .next-demo	Nền 1f2bedb với DB demo riêng; chưa phải deployment production
MC-05	node scripts/demo.cjs --no-start; node tests/demo-db.cjs; reproducibility-evidence.json	MySQL 8.0.46: seed, chạy lại, CHECK/FK/unique và từ chối DB không thuộc công cụ đều đạt	Có ghi kiểm thử trên DB riêng; không xác minh chuỗi migration lịch sử
MC-06	node scripts/check-security-http.cjs http://127.0.0.1:3001; hồ sơ security/evidence/	8/8 đạt trước đồng bộ: headers, origin lạ/preflight 403, cùng origin hoạt động	Nền 809f9bc; chưa chạy lại HTTP sau đồng bộ; không phải chứng nhận bảo mật

Mã	Lệnh / bằng chứng	Kết quả quan sát	Phạm vi kết luận
MC-07	npm run api:docs:check; documentation/evidence/api-http.json	Spec sinh lại/khớp source: 28 đường dẫn/45 thao tác; HTML/JSON 200 ở lần HTTP trước đồng bộ	Spec nền 1f2bedb; HTTP nền 809f9bc; chưa thử đủ 45 nghiệp vụ
MC-08	documentation/evidence/demo-http.json	5/5 phép HTTP đạt trước đồng bộ	Nền 809f9bc; chuyến seed, ADMIN/CUSTOMER, profile và chặn khách ở API admin
MC-09	docs/chapter7/evidence/runtime-pages.json; ảnh Hình 7.3–7.4	Chrome headless đọc trang chủ và /trips trên demo trước đồng bộ	Ảnh nền 809f9bc ngày 06/11/2026; chưa chụp lại; không thao tác đặt/thanh toán

Bảng 7.5. Kết quả có bằng chứng ngày 06/11/2026 và phạm vi kết luận

Kết quả chính trước khi push là npm test: 162/162 ca đạt, 0 lỗi và 0 bỏ qua, khoảng 6,30 giây (6.304,39 ms). Bộ này đã bao gồm test demo, bảo mật, hook tải dữ liệu và tài liệu API trong tests/*.test.cjs; không cộng lại các bộ con 11 hoặc 61 ca. Tám phép HTTP bảo mật và năm phép HTTP demo là phạm vi riêng, có thể dùng chung endpoint. Thời gian runner không phải latency API, throughput hoặc thời gian hoàn tất một lượt đặt.

Lần thử cũ trên nền 809f9bc chạy toàn bộ tệp trong một tiến trình chung chỉ đạt 153/159, được giữ tại documentation/evidence/unit-in-process-failed.log và verification-before-sync.json; fixture biến môi trường và module cache có thể ảnh hưởng giữa các tệp. Đây là lần thử lịch sử, không phải 6 lỗi của bộ 162 ca hiện tại và không được cộng vào kết quả cách ly từng tệp. Khi sandbox chặn spawn EPERM, cần chạy npm test trong môi trường cho phép tiến trình con; không khuyến nghị --test-isolation=none cho toàn bộ source hiện tại.

Các ca kiểm tra bao gồm phân quyền, nguồn yêu cầu, token phiên, hồ sơ/lịch sử, tra cứu có giới hạn, dữ liệu tìm chuyến, chuẩn hóa ID/ghế, retry transaction, thống kê, QR/Demo, cập nhật realtime, an toàn dựng demo và độ khớp tài liệu API. Ba ca hook mới kiểm tra bỏ kết quả request đã hủy, giữ dữ liệu khi lỗi/tải lại và bỏ lỗi muộn sau unmount. Kiểm thử demo MySQL xác minh seed, CHECK/FK/unique, bảo toàn dữ liệu khi chạy lại và từ chối database không thuộc công cụ. Nó chưa thay thế kiểm thử cuộc đua đặt/thanh toán đồng thời hoặc kiểm chứng chuỗi migration lịch sử.

Hồ sơ bảo mật đối chiếu đủ 10 nhóm OWASP Top 10:2025 ở mức nhận biết, ghi biện pháp và phần chưa kiểm chứng. Có test escaping XSS trên component thật, đối số Prisma và chính sách origin; tám kiểm tra HTTP trước đồng bộ xác minh header/preflight thực trên nền 809f9bc. Chưa chạy lại HTTP sau đồng bộ. CSP mới giới hạn base-uri, object-src, frame-ancestors và form-action, chưa có script-src nonce/hash. Chưa có số liệu độ bao phủ, latency/throughput, Lighthouse hoặc chứng nhận WCAG/kiểm toán bảo mật.

7.2.5. Cài đặt và triển khai để tái lập

package-lock.json khóa Next.js 16.3.5, React/React DOM 19.2.8, Prisma 6.19.3, TypeScript 5.9.3, Tailwind CSS 4.3.3, Zod 4.6.5, React Hook Form 7.88.0 và ESLint 9.39.5. Next yêu cầu Node.js từ 20.9; hướng dẫn demo chọn Node.js 22 trở lên và MySQL local. Môi trường đã kiểm tra thực tế là Node.js 24.22.1 với MySQL 8.0.46; chưa suy rộng cho mọi bản MySQL/MariaDB.

Đường tái lập đã kiểm chứng dùng scripts/demo.cjs: máy cần có Node/npm, dịch vụ MySQL local và tài khoản có quyền tạo database. Script nhận DEMO_SERVER_URL hoặc lấy thông tin máy chủ/tài khoản từ DATABASE_URL trong .env, nhưng luôn chọn một database demo riêng. Từ thư mục dự án, chạy một lệnh:

```
npm run demo
```

Nếu chưa có .env hoặc muốn chỉ rõ máy chủ local, đặt biến môi trường DEMO_SERVER_URL theo mẫu dưới đây bằng terminal hoặc cơ chế quản lý cấu hình. Đây là placeholder, không phải bí mật lấy từ môi trường đang dùng:

```
DEMO_SERVER_URL="mysql://DB_USER:DB_PASSWORD@127.0.0.1:3306/mysql"
```

Lệnh demo tự cài theo lockfile khi thiếu dependency, kiểm tra/tạo Prisma Client, khởi tạo snapshot schema và dữ liệu rồi chạy ứng dụng tại http://127.0.0.1:3001. Bản đã khớp schema được dùng lại để tránh ghi đè DLL đang bị Windows khóa. SESSION_SECRET riêng sinh ngẫu nhiên trong bộ nhớ; QR dùng dữ liệu minh họa và xác nhận mô phỏng, không chuyển tiền thật. Nhánh tự npm ci từ node_modules rỗng chưa được chạy trong hồ sơ này.

Nếu chỉ chuẩn bị môi trường mà chưa mở server, dùng:

```
npm run demo:prepare
```

Dữ liệu mẫu gồm ADMIN 0911000001, CUSTOMER 0911000002 hoặc customer@example.test, mật khẩu demo mặc định Demo@12345 (có thể cấu hình DEMO_PASSWORD trước lần tạo mới). Mật khẩu được băm scrypt. Có sáu chuyến tương lai, hai xe, một nhân viên và một vé lịch sử. Chạy lại chỉ dùng database có marker sở hữu/schema đúng, không reset hoặc seed trùng; tên đã tồn tại không thuộc công cụ bị từ chối. Khi cần demo mới dùng --database sgu_booking_demo_<suffix>.

Đường demo dùng .next-demo, tách khỏi thư mục .next của ứng dụng thường. Các lệnh chất lượng và production thông thường dưới đây cần môi trường cấu hình phù hợp; hồ sơ hiện tại ghi riêng bản build với SGU_DEMO=true. npm run build kiểm tra độ khớp OpenAPI trước khi gọi Next:

```
npm run api:docs:check
npm run lint
npm run typecheck
npm test
npm run build
npm start
```

npm start cần bản build production tương ứng đã hoàn tất. Source dùng next/font/google nên cài/build có thể cần Internet để tải font; cấu hình production cần HTTPS cho cookie Secure. next.config.ts hiện có allowedDevOrigins, header bảo mật và distDir riêng cho demo. Chưa có Docker, output standalone hoặc bằng chứng triển khai cloud. Hướng dẫn đầy đủ: documentation/reproducibility.md và documentation/security/README.md.

Database ứng dụng cũ có luồng nâng cấp riêng; không dùng đường demo để nâng cấp dữ liệu đang sử dụng. README ghi nhận lỗi MySQL 3823 của một migration lịch sử có CHECK tham chiếu cột FK ON UPDATE CASCADE. Các lệnh nâng cấp dưới đây cần được đối chiếu trên bản sao phù hợp trước khi áp dụng:

```
npm run db:upgrade
npx prisma generate
npm run dev
```

Script nâng cấp kiểm tra dữ liệu, ghi backup schema và rows trong .local-backups, bổ sung cấu trúc thiếu, kiểm tra baseline và triển khai migration; lịch sử khác bị từ chối. Hồ sơ demo không sửa checksum migration đã áp dụng: chỉ db push vào database mới do công cụ tự tạo, sau đó bổ sung năm CHECK và dùng FK booking.tripId ON UPDATE RESTRICT. Đường snapshot đã chạy được không chứng minh lịch sử migrate deploy đã được giải quyết; backup cũng chưa tương đương có script restore tự động.

db:payments và db:trips xử lý riêng bảng Payment hoặc cột mã chuyến còn sót trong database import; không thay thế nâng cấp toàn schema. prisma.config.ts trỏ đến prisma/admin-migrations, còn prisma/migrations là lịch sử cũ. Các lần tạo/ghi dữ liệu trong hồ sơ mới chỉ dùng database demo hoặc database kiểm thử tách riêng; database ứng dụng và .env không bị chỉnh sửa.

7.2.6. Hồ sơ minh chứng và các bước kiểm tra bổ sung

Hồ sơ mới được lưu trong documentation/ để có thể đi cùng source; verification.json ghi nền commit, source/log hash và các phạm vi đã chạy; log trước khi push có tiền tố pre-push-. verification-before-sync.json giữ nguyên hồ sơ nền 809f9bc, bao gồm lần thử 153/159 và kết quả 159/159 cũ. Riêng nội dung, bản Word/PDF, sơ đồ và ảnh Chương 7 nằm trong docs/chapter7. Hash là của các tệp được chọn, không phải một bản niêm phong toàn repository. Sơ đồ truy vết mã nguồn và ảnh runtime là hai loại minh chứng khác nhau; ảnh chụp trang không thay cho request/response hoặc dữ liệu trước–sau của giao dịch.

Nhóm bằng chứng	Đường dẫn	Ý nghĩa
Tổng hợp nguồn/kết quả	documentation/evidence/verification.json	Nền commit, phạm vi, lệnh, exit code và SHA-256 tệp được chọn
Unit và kiểm tra tĩnh	documentation/evidence/pre-push-unit.log; pre-push-typecheck.log; pre-push-lint.log	162 ca đạt; TypeScript/ESLint trên nền 1f2bedb

Nhóm bằng chứng	Đường dẫn	Ý nghĩa
Build và hồ sơ cũ	documentation/evidence/pre-push-build.log; pre-push-build-result.json; verification-before-sync.json; unit-in-process-failed.log	Build trước khi push; hồ sơ cũ giữ 159/159 và lần thử 153/159 trên nền 809f9bc
Tái lập và MySQL	documentation/reproducibility.md; reproducibility-evidence.json; evidence/demo-http.json	Seed và ràng buộc trên DB riêng, bảo toàn khi chạy lại, 5 phép HTTP
Bảo mật	documentation/security/README.md; documentation/security/evidence/	Đối chiếu OWASP Top 10:2025; kết quả logic/HTTP và giới hạn
Đặc tả API	documentation/api/; documentation/evidence/api-http.json	OpenAPI sinh lại sau đồng bộ; HTTP HTML/JSON thuộc lần kiểm tra trước đồng bộ
Hồ sơ Chương 7	docs/chapter7/content.md; assets/; evidence/source-manifest.json; runtime-pages.json	Nội dung/sơ đồ theo source hiện tại; hai ảnh nền 809f9bc trước đồng bộ và metadata ảnh

Bảng 7.6. Danh mục hồ sơ mới và vị trí minh chứng

Các bộ kiểm thử database bổ sung vẫn có trong repository: test:admin-db, booking-fixes-db.cjs, payment-db.cjs và payment-setup-db.cjs. Từng bộ có cách tạo/dọn database tạm riêng; chưa có log PASS mới cho toàn bộ những lệnh này trong hồ sơ Chương 7. Không dùng kết quả demo-db.cjs để khẳng định đã thử cuộc đua đặt cùng ghế, hai xác nhận thanh toán đồng thời hoặc chuỗi migration lịch sử trên database ứng dụng.

Các script *.browser.cjs không thuộc npm test và cần xem điều kiện riêng trước khi chạy. Một số dùng Playwright/fixture API; booking-fixes.browser.cjs dùng Chrome CDP và Node, không cần cài Playwright. Những script có ghi dữ liệu phải dùng database thử. Ảnh của chương này được chụp trước đồng bộ trên nền 809f9bc bằng Chrome headless, chỉ đọc trang chủ và danh sách chuyến, không đăng nhập, chọn ghế hoặc gửi giao dịch; chưa tương đương chạy toàn bộ browser/E2E suite.

Khi mở rộng minh chứng, cần lưu đồng thời ảnh màn hình, request/response hoặc sự kiện SSE, dữ liệu trước–sau theo tripId/PNR và log gắn phiên bản/thời điểm. Chỉ công bố số đo hiệu năng hoặc tiếp cận sau khi có công cụ, cấu hình và kết quả thực tế. Mục đích của bảng kết quả là xác định đúng mức hoàn thành, không biến giới hạn của mock hoặc ảnh trang thành bằng chứng giao dịch end-to-end.

7.2.7. Hạn chế của kết quả hiện tại

Các giới hạn dưới đây được rút từ source và phạm vi kiểm tra, đồng thời xác định những nội dung chưa được công bố là hoàn thành:

• Realtime chọn tạm và viewer nằm trong RAM từng tiến trình; chưa có pub/sub nhiều instance, lưu lịch sử sự kiện, hàng đợi thao tác offline hoặc CRDT. Polling database không làm cho các Map lựa chọn được chia sẻ giữa các instance.

• UI đã đọc trạng thái giữ ghế nhưng chưa tự gọi POST để tạo SeatHold khi bấm ghế. Worker hết hạn vé online đã có, song cần tiến trình Node đang chạy và xử lý theo đợt; không cam kết giải phóng đúng giây hết hạn hoặc hoàn tiền.

• VietQR và xác nhận Demo chưa kiểm chứng tiền vào ngân hàng, webhook hoặc hoàn tiền. Tra cứu vé đã yêu cầu PNR + điện thoại, nhưng /success và các API thanh toán vẫn dựa vào PNR; chưa gắn kiểm soát chủ sở hữu theo tài khoản cho toàn bộ nhóm này.

• Bộ lọc giờ/ngày đã chuyển lên server theo UTC+7 và áp dụng trước phân trang. Nhóm xe hiện quy ước theo capacity 22, 34 hoặc sức chứa khác, chưa lấy loại xe từ một quan hệ xe–chuyến riêng.

• Demo snapshot và ràng buộc đã chạy trên MySQL 8.0.46; lịch sử migration có lỗi tương thích cần xử lý riêng. Chưa kiểm thử Docker hoặc nhánh cài dependency từ thư mục rỗng, và chưa có quy trình khôi phục backup tự động hoàn chỉnh.

• Đã có unit/handler, HTTP và kiểm thử ghi trên database demo, nhưng chưa có full browser/E2E, cuộc đua đặt/thanh toán trên MySQL, đo tải, audit WCAG hoặc kiểm toán bảo mật toàn hệ thống. Chưa có public deployment, GitHub Actions CI, minh chứng đầy đủ review chéo/bảo vệ main và mục AI Disclosure trong hồ sơ.